

Extended Abstract

Motivation Large language models (LLM) benefit from generating longer outputs for tasks like reasoning and coding Wei et al. (2023); Snell et al. (2024), but this comes at an increase in computational cost. Models trained from RLHF suffer from verbose responses due to biased reward models that prefer longer responses regardless of quality Bu et al. ([n. d.]). While recent work addresses output efficiency Fatemi et al. (2025) by training on shorter responses, it risks discarding valuable reasoning patterns in longer outputs. We aim to balance informativeness and efficiency through a length-penalty annealing strategy that encourages learning from longer responses early in training and promotes conciseness in the later stages.

Method We formulate efficiency optimization as a multi-objective task balancing task-specific reward and brevity. For a given prompt s and response a , we define a scalarized reward function:

$$\hat{R}(s, a) = (1 - \alpha) \cdot T(s, a) - \alpha \cdot L(s, a) \quad (1)$$

where $T(s, a)$ is a task-specific reward and $L(s, a)$ is a normalized length penalty based on output token count. The trade-off parameter $\alpha \in [0, 1]$ is annealed over training time using either linear or exponential schedules, starting from α_0 and gradually increasing to α_T to encourage concise responses. An example of the annealing schedules are shown in Figure 1. This annealing strategy is integrated into two algorithms. In iterative DPO, we relabel a preference dataset at each epoch using $\hat{R}(s, a)$ and apply the DPO loss to updated preferences. In iterative RLOO, we sample responses on-policy, compute leave-one-out advantages using $\hat{R}(s, a)$, and update the policy.

Results We fine-tuned the Qwen/Qwen2.5-0.5B base model on two tasks: ULTRAFEEDBACK for instruction-following and COUNTDOWN for mathematical reasoning, using both iterative DPO and RLOO with fixed, linear, and exponential annealing. Across experiments, annealing strategies consistently reduced output length while maintaining or improving task performance. For ULTRAFEEDBACK, annealing increased the task performance by 10% while maintaining similar response length to using fixed objective weights. These results are shown in Figure 2. For RLOO, we observed similar performance across the different annealing strategies.

Conclusion Our results demonstrate the effectiveness of annealing, especially when employing exponential schedules in instruction-following tasks. Future research may investigate adaptive annealing using validation performance and assessing the method’s scalability to larger LLMs.

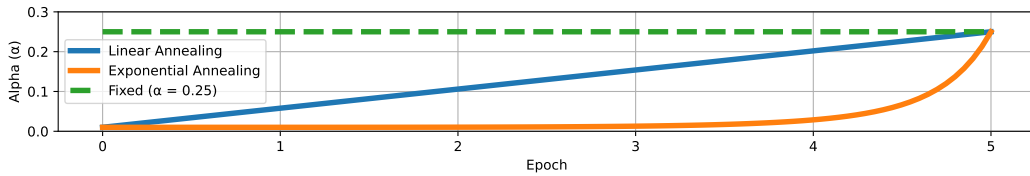


Figure 1: Different annealing schedules for $\alpha(t)$. For the both annealing strategies, we set $\alpha_0 = 0.01$ and $\alpha_T = 0.25$. For exponential annealing, we set $s = 5$.

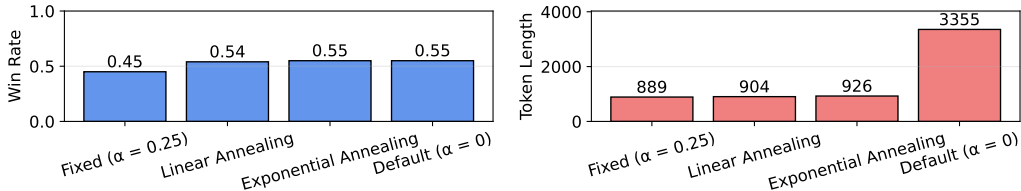


Figure 2: Performance on the ULTRAFEEDBACK task for the proposed iterative DPO algorithm for different annealing schedules.

Training LLMs to Respond Efficiently with Reinforcement Learning

Abdulaziz Alharbi

Department of Electrical Engineering
Stanford University
aalharbi@stanford.edu

Abstract

With the growing popularity of scaling test-time compute to enhance the performance of large language models (LLMs), concerns about the computational efficiency of these methods have increased. In this project, we address the problem of minimizing the number of output tokens in LLMs by formulating it as a multi-objective optimization problem that balances task success and response length. While existing approaches often impose fixed token budgets or restrict training data to short responses, we propose an alternative: an annealing strategy in which objective weights are gradually adjusted during training. This allows the model to benefit from learning with longer responses in early training stages while progressively emphasizing brevity in later stages, akin to a curriculum learning. We adapt the DPO and RLOO algorithms to incorporate this annealing approach. On the ULTRAFEEDBACK task, our method improves task success by 10% while maintaining response lengths comparable to a baseline with fixed objective weights. However, in sparse reward settings such as the COUNTDOWN task, we observe only marginal improvements from weight annealing.

1 Introduction

Scaling the test-time compute of large language models (LLM) has led to remarkable success in tasks such as mathematical reasoning and coding Snell et al. (2024); Brown et al. (2024). Approaches that increase test-time compute often employ techniques such as Chain of Thought (CoT) reasoning, which typically require the LLM to generate significantly more tokens than standard models Wei et al. (2023). Despite their advantages, this substantial increase in output length results in greater computational complexity, potentially limiting their applicability in time- or resource-constrained settings.

Many of these models are trained using reinforcement learning from human feedback (RLHF), which depends on human supervision to compute the reward signal. However, studies have shown that biased reward models tend to assign higher rewards to longer responses, even when these are unnecessarily verbose for the task at hand Bu et al. ([n. d.]).

Recently, there has been growing interest in improving the output token efficiency of LLMs Fatemi et al. (2025). While these techniques can be effective, they typically involve fine-tuning the model on a subset of short responses. This may prevent the model from learning useful reasoning patterns present in longer, more informative outputs.

Motivational Example: As a motivational example, we analyzed the reward distribution as a function of response length for two tasks: the *Ultrafeedback* dataset Cui et al. (2024), which evaluates an LLM’s ability to produce helpful and harmless content, and the *CountDown* task Gandhi et al. (2024), which assesses mathematical reasoning and logical capabilities. The results, shown in Figure 3, reveal

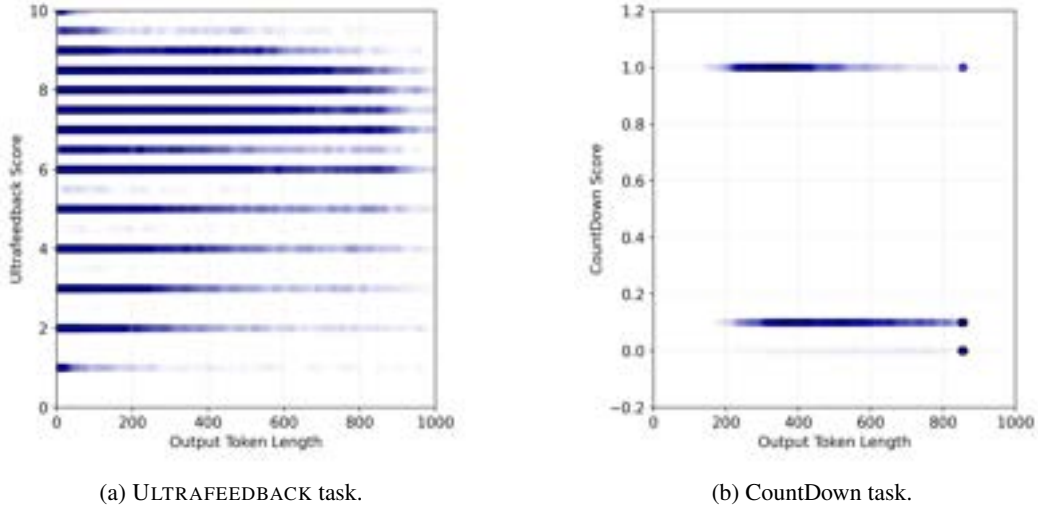


Figure 3: Distribution of the rewards as a function of the number of output tokens. Each dot denote a single example. For the COUNTDOWN task, the responses were generated using on-policy samples from a fine-tuned Qwen/Qwen2.5-0.5B model on the WarmStart dataset.

that longer responses in the ULTRAfeedback task tend to receive a high concentration of high rewards, while shorter responses exhibit a more dispersed reward distribution. In contrast, for the COUNTDOWN task, longer responses generally receive lower rewards than shorter ones.

Our hypothesis is that fine-tuning an LLM with an objective that strongly penalizes response length may unintentionally bias the model against producing the useful reasoning and logical behaviors often found in longer responses. To address this, we propose a length-penalty annealing strategy, which can be viewed as a form of curriculum learning. Early in training, the model is not penalized for response length, allowing it to learn from longer, high-reward responses. As training progresses, the penalty for length is gradually increased to encourage brevity. This approach aims to strike a balance between learning from informative long responses and generating concise outputs.

2 Related Work

Multi-Objective Optimization for LLMs: Numerous works have been proposed in the literature to develop methods for optimizing multi-objective rewards in LLMs. Liu et al. (2024) used the dual gradient descent algorithm to optimize a constrained variant of Direct Preference Optimization (DPO), aiming to optimize preferences while maintaining safety requirements. Xu et al. (2024) proposed Constrained Generative Policy Optimization (CGPO), a multi-objective RLHF approach that balances several objectives in LLM fine-tuning using a mixture of judge models specific to each objective. Yang and Li (2023) proposed an approach that uses evolutionary algorithms to optimize instruction quality and the length of input prompts to LLMs. They employed ChatGPT to serve as a mutation/crossover operator for evolving instruction prompts.

Optimizing LLMs’ Response Efficiency: There has been increasing interest in developing methods to improve the response efficiency of LLMs. Han et al. (2025) introduces a framework that dynamically estimates token budgets based on the complexity of reasoning tasks and incorporates them into the prompt. Xia et al. (2025) proposes an approach that analyzes the semantic importance of tokens within CoT outputs and uses the importance weights to fine-tune an LLM to learn to skip non-essential tokens. Butcher et al. (2025) introduces an approach that uses a secondary length-difference positional encoding (LDPE), which counts down the remaining tokens and is added to the model’s input embeddings during training to encourage brevity. While these approaches are based on prompt engineering and supervised fine-tuning (SFT) techniques, some works have applied reinforcement learning (RL) to this problem. Chen et al. (2025) uses problem difficulty to dynamically adapt token penalty strength in the reward function. Fatemi et al. (2025) introduces a two-stage RL framework

that first fine-tunes LLMs using task-specific objectives and then uses RL to fine-tune for response brevity.

Multi-Objective Optimization with Reward Annealing: While there has been some work in the literature developing RL methods that assign dynamic weights to different objectives in the reward, such as the works in Yang et al. (2024) and Huang et al. (2021), little research has explored reward annealing in RL. Freitag et al. (2025) proposed an approach in which an agent optimizes a simplified reward to learn the basic task. Once the policy converges on the main goal, based on the performance of the Q -function, training switches to the full composite reward.

2.1 Contribution

To the best of our knowledge, no prior work has considered annealing multi-objective reward weights over time for LLMs to optimize for brevity. The closest work in the literature to ours is the recently published study by Chen et al. (2025). However, in that work, the length penalty weights are assigned dynamically rather than using an annealing-based approach.

3 Method

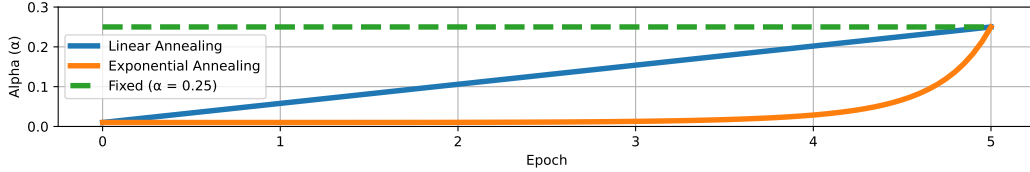


Figure 4: Different annealing schedules for $\alpha(t)$. For the both annealing strategies, we set $\alpha_0 = 0.01$ and $\alpha_T = 0.25$. For exponential annealing, we set $s = 5$.

3.1 Multi-objective Rewards

We formulate the problem of optimizing response efficiency as a multi-objective optimization task. Given an input prompt s and a corresponding response a from an LLM assistant, we define a scalarized multi-objective reward function $\hat{R}(s, a)$ as follows:

$$\hat{R}(s, a) = (1 - \alpha) \cdot T(s, a) - \alpha \cdot L(s, a) \quad (2)$$

Here, $T(s, a)$ represents a task-specific reward (e.g., the verifier score in the COUNTDOWN task), while $L(s, a)$ is a penalty term that discourages verbosity. Specifically, we define the length penalty $L(s, a)$ as the number of output tokens generated by the LLM assistant. The parameter $\alpha \in [0, 1]$ controls the trade-off between the task reward and the length penalty.

Since most task-specific rewards we consider lie in the range $[0, 1]$, tuning α becomes challenging if the length penalty dominates the reward function. To address this, we normalize $L(s, a)$ by the maximum number of output tokens allowed during training, ensuring that both objectives are scaled to the $[0, 1]$ range.

3.2 Reward Weight Annealing

To prevent the model from being overly biased against the valuable information often found in longer responses, we propose annealing the reward weight α over training time. Specifically, we define $\alpha(t)$ as a function of the current training epoch t . The value of α is gradually increased from an initial value α_0 to a final value α_T over the course of T total training epochs, according to a predefined annealing schedule.

Typically, α_0 is set close to zero, allowing the model to benefit from longer, informative responses during the early stages of training. As training progresses, α increases, encouraging the model to produce more concise outputs. This approach can be viewed as a form of curriculum learning.

We consider two types of annealing schedules: linear and exponential. Figure 4 highlights the behavior of the different annealing strategies considered.

Linear Annealing. The linear schedule is defined as:

$$\alpha(t) = \alpha_0 + (\alpha_T - \alpha_0) \cdot \frac{t}{T} \quad (3)$$

Exponential Annealing. The exponential schedule is defined as:

$$\alpha(t) = \alpha_0 \cdot \left(\frac{\alpha_T}{\alpha_0} \right)^{\left(\frac{t}{T} \right)^s} \quad (4)$$

Here, s is a shaping parameter that controls the rate of exponential growth. Higher values of s result in slower increases in α , providing greater emphasis on the task-specific reward $T(s, a)$ for a longer portion of the training process.

Algorithm 1 Iterative DPO

Require: Policy π , Reference Policy π_{ref} , Preference dataset $\mathcal{D}$ with task-specific reward $T(s, a)$ values.

- 1: **for** epoch t **do**
- 2: Update $\alpha(t)$ and Relabel $\mathcal{D}$ using Eq. (2) as preference dataset $\hat{\mathcal{D}}$
- 3: **for** batch b in $\hat{\mathcal{D}}$ **do**
- 4: Take gradient step ℓ_{DPO}
- 5: **end for**
- 6: **end for**

Algorithm 2 Iterative RLOO

Require: Policy π , Task-specific reward $T(s, a)$

- 1: **for** epoch t **do**
- 2: Update $\alpha(t)$
- 3: **for** step i **do**
- 4: Sample k responses and compute rewards using Eq. (2)
- 5: Take gradient step ℓ_{RLOO}
- 6: **end for**
- 7: **end for**

3.3 Policy Optimization

We adapt two commonly used algorithms to optimize the multi-objective reward defined in Eq. 2: DPO and Reinforce Leave-One-Out (RLOO).

Iterative Direct Preference Optimization. We extend the DPO algorithm to incorporate the proposed annealing strategy. Given a parameterized policy π_θ , a reference policy π_{ref} , a task-specific reward $T(s, a)$, and a preference dataset $\mathcal{D}$, we update the value of α at the beginning of each training epoch according to the annealing schedule $\alpha(t)$.

Next, we relabel the original preference dataset $\mathcal{D}$ using the scalarized multi-objective reward in Eq. 2, resulting in a new preference dataset $\hat{\mathcal{D}}$. We then apply the standard DPO loss to $\hat{\mathcal{D}}$, thereby enabling the model to progressively balance between task performance and brevity as training proceeds using the dynamic preference dataset. We denote this algorithm as iterative DPO.

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \hat{\mathcal{D}}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]. \quad (5)$$

It is important to note that the functional form of the task-specific reward is not required for the iterative DPO algorithm, as we only relabel the existing responses in the dataset $\mathcal{D}$ and do not generate any additional on-policy samples. Specifically, we only require access to the scalar task-specific

reward values associated with each response. The complete details of the algorithm are provided in Algorithm 1.

Iterative Reinforce Leave-One-Out. The proposed annealing strategy can be seamlessly integrated into existing policy gradient methods. For simplicity, we focus on the Reinforce Leave-One-Out (RLOO) algorithm. Given a policy π and a functional form of the task-specific reward $T(s, a)$, we update the value of α at the beginning of each training epoch according to the annealing schedule $\alpha(t)$. To compute the advantage for samples generated by the current policy π , we use the scalarized multi-objective reward defined in Eq. 2. Once the advantages are computed using this reward formulation, we apply the standard RLOO loss to optimize the policy using the labeled responses. We denote this algorithm as iterative RLOO. The complete details of the algorithm are provided in Algorithm 2.

$$\frac{1}{k} \sum_{i=1}^k \left[\hat{R}(y_{(i)}, x) - \frac{1}{k-1} \sum_{j \neq i} \hat{R}(y_{(j)}, x) \right] \nabla \log \pi(y_{(i)} | x) \quad \text{for } y_{(1)}, \dots, y_{(k)} \stackrel{i.i.d.}{\sim} \pi_{\theta}(\cdot | x) \quad (6)$$

4 Experimental Setup

We evaluated the proposed method on the ULTRAFEEDBACK and COUNTDOWN tasks by fine-tuning the Qwen/Qwen2.5-0.5B base model. We compared three variants of the multi-objective reward in Eq. 2: fixed weighting, linear annealing, and exponential annealing against a baseline without a length penalty. Performance is measured using task-specific rewards and output token count. For evaluation, we use VLLM for sampling and set the maximum number of tokens to 4096.

4.1 Ultrafeedback

We evaluate the proposed methods on the ULTRAFEEDBACK task, which measures the instruction-following capabilities of LLMs and assesses their ability to produce both useful and harmless content.

Warm-start. We initialize from the Qwen/Qwen2.5-0.5B base model and perform SFT on the SmolTalk dataset Allal et al. (2025). The model is trained for one epoch using AdamW with a learning rate of 2×10^{-5} and linear warmup over the first 10% of training steps. We select the checkpoint with the lowest validation loss. Training is conducted with a batch size of 4 and 16 gradient accumulation steps. We clip the maximum number of tokens per example to 1024.

Reward Modeling. To train the iterative RLOO model, we require access to the functional form of the task-specific reward $T(s, a)$. However, the ULTRAFEEDBACK dataset provides human feedback rather than an explicit reward function. To address this, we train a reward model using the Bradley-Terry objective on a subset of 60,000 examples from the ULTRAFEEDBACK dataset. For the reward model architecture, we use the Qwen/Qwen2.5-0.5B base model with a linear projection head. The model is trained for one epoch using AdamW with a learning rate of 2×10^{-5} and cosine warmup over the first 10% of steps. As in the warm-start phase, we use a batch size of 4 and 16 gradient accumulation steps.

Training Hyperparameters. For iterative DPO, we set the parameter $\beta = 0.1$ and train the models for 5 epochs using a batch size of 4 with 16 gradient accumulation steps. Optimization is performed using AdamW with a learning rate of 1×10^{-5} and cosine warmup over the first 10% of training steps. We use 2,000 examples from the ULTRAFEEDBACK dataset as the preference data for these experiments.

For iterative RLOO, we set the number of samples per update to $k = 8$ and train the models for 5 epochs with a batch size of 1 and 16 gradient accumulation steps using the learned reward model. We cap the maximum number of generated tokens at 1024. Due to the high time complexity of generating on-policy samples, we limit training to 100 steps per epoch. The model is optimized using AdamW with a learning rate of 2×10^{-5} and cosine warmup over the first 10% of steps.

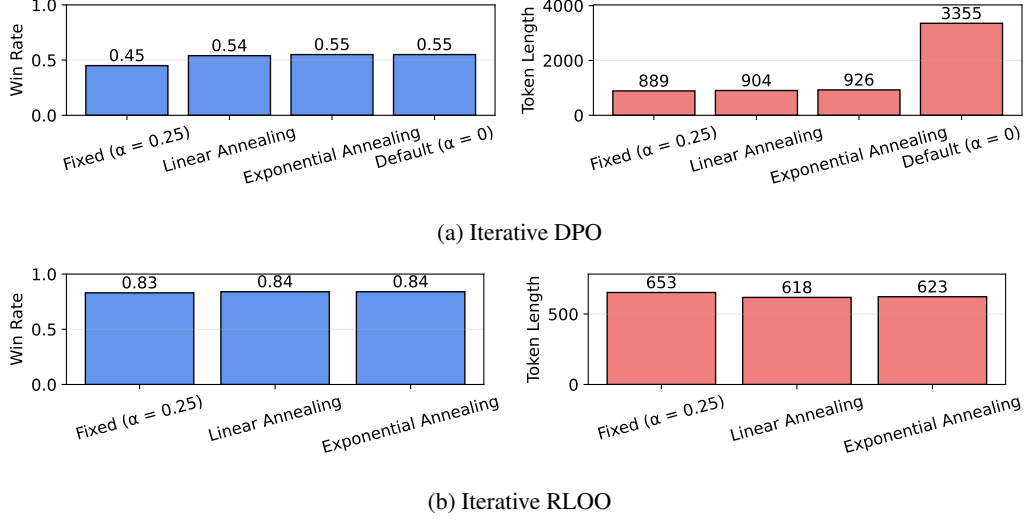


Figure 5: Performance comparison on the ULTRAfeedback task for different annealing strategies for (a) Iterative DPO, and (b) Iterative RLOO. Unfortunately, we were not able to run the default scenario for this experiment for iterative RLOO as we ran out of computational budget.

4.2 CountDown

We also evaluate the proposed methodology on the COUNTDOWN task, which assesses the mathematical reasoning and logical capabilities of LLMs.

Warm-start. We begin from the warm-start initialization used for ULTRAfeedback and apply SFT on the WarmStart dataset Gandhi et al. (2025). Due to the relatively small size of the dataset, we train the model for 5 epochs using AdamW with a learning rate of 2×10^{-5} and linear warmup over the first 10% of training steps. The training set consists of 1,000 examples, with an additional 200 examples used for validation.

Training Hyperparameters. For iterative DPO, we set parameter $\beta = 0.1$ and train the models for 5 epochs using a batch size of 4 with 16 gradient accumulation steps. We use AdamW with a learning rate of 1×10^{-5} and cosine warmup over the first 10% of training steps. The preference dataset $\mathcal{D}$ consists of 2,000 on-policy samples generated from the warm-start checkpoint using a sampling temperature of 0.8.

For iterative RLOO, we use $k = 8$ samples and train the models for 5 epochs with a batch size of 1 and 16 gradient accumulation steps. We use the verifier-based reward function provided by the task. As in ULTRAfeedback, we cap the maximum number of generated tokens at 1024 and use AdamW with a learning rate of 2×10^{-5} and cosine warmup over the first 10% of training steps.

4.3 Annealing Schedules

For all annealing experiments in both iterative DPO and iterative RLOO, we use the same annealing parameters: $\alpha_0 = 0.01$, $\alpha_T = 0.25$, and shaping parameter $s = 0.25$.

5 Results and Discussion

Quantitative Evaluation - Ultrafeedback: Figure 5 presents results on the ULTRAfeedback task using the SFT-initialized model as the reference. For iterative DPO, the proposed annealing strategies achieve competitive task performance while significantly reducing output length. Notably, the exponential schedule matches the default model win rate while reducing token count by 72%. We also observed that the default model occasionally produced outlier responses that hit the maximum token limit.

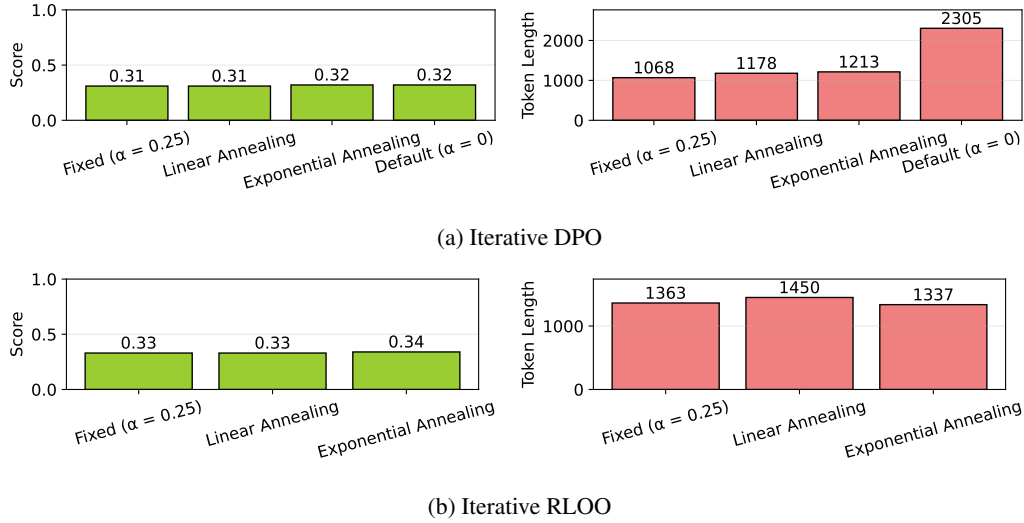


Figure 6: Performance comparison on the COUNTDOWN task for different annealing strategies for (a) Iterative DPO, and (b) Iterative RLOO. Unfortunately, we were not able to run the default scenario for this experiment for iterative RLOO as we ran out of computational budget.

The exponential schedule slightly outperforms the linear variant by 1% in win rate, with only a 2.4% increase in token count. While the fixed schedule results in the fewest number of output tokens, it decreases the win-rate below 50%. Compared to using a fixed reward weight, annealing improves win rate by approximately 10%.

From Figure 5, iterative RLOO generally outperforms iterative DPO in terms of task performance. Among the annealing strategies, performance differences are minor, with linear annealing achieving the highest win rate while also producing the shortest responses.

Quantitative Evaluation – COUNTDOWN. Figure 6 presents results for the COUNTDOWN task. For iterative DPO, incorporating the length penalty reduces the number of output tokens by approximately 47%. As in the ULTRAfeedback task, removing the length penalty led to more outlier cases where the model continued generating until the maximum token limit. Overall, the task scores remain relatively consistent across models. The fixed schedule achieved the lowest token count, though with a slight drop in performance. Overall, the exponential annealing strategy yields the highest performance while increasing the number of tokens by 14% compared to the fixed schedule.

For iterative RLOO, we observe a similar trend: different annealing strategies yield comparable scores. However, iterative RLOO trained models generally outperform the iterative DPO ones in terms of task score, although with longer outputs. Additionally, RLOO models exhibit greater variability in response length across examples, which might be why we see exponential annealing outperforming the fixed schedule.

Qualitative Analysis: We also conducted a qualitative analysis to better understand model behavior. For the ULTRAfeedback task, we compared responses from different models, focusing on the reduction in response length achieved by the iterative DPO model with exponential annealing, relative to the default model without a length penalty. We observed that the default model often includes unnecessary details not requested by the user, whereas the iterative DPO model typically responds more concisely and directly. This difference is illustrated in Example #5 below.

For the COUNTDOWN task, we analyzed examples generated by iterative RLOO using both fixed and linear annealing schedules to understand the increase in output length with the linear schedule. We observed that the fixed schedule generally produces more efficient responses, using fewer reasoning steps compared to the linear schedule, which tends to generate more verbose explanations. This behavior is illustrated in Example #7 below. While both models arrive at the correct final answer, the fixed schedule does so with fewer steps.

6 Conclusion

In this project, we proposed an approach for optimizing the response efficiency of LLMs by minimizing the number of output tokens through a novel annealing strategy. Our method begins by prioritizing task success in the early stages of training and gradually shifts toward optimizing for brevity, enabling a balanced trade-off between task success and conciseness. Our results highlight the benefits of weight annealing overall, and of exponential schedules in particular—especially for instruction-following tasks.

Future work could explore the use of dynamic annealing schedules, where the value of α is adjusted based on validation set performance rather than following a fixed schedule. Additionally, further research could investigate applying and evaluating the proposed methodology on larger-scale LLMs to assess its scalability and effectiveness in more complex settings.

7 Team Contributions

All of the work presented in this project was completed solely by me.

Changes from Proposal The initial proposal aimed to address the research question: *Can we train LLMs using multi-objective rewards with RL to efficiently query other LLMs for assistance during generation?*. The original proposal’s main theme revolved around multi-objective optimization. However, due to limited computational resources, we had to shift our focus to optimizing the response efficiency of LLMs.

Example #5: Ultrafeedback Qualitative Samples

Input Prompt

What is the capital city of Australia?

Default Model with Standard DPO ($\alpha = 0$)

The capital city of Australia is Canberra, a city located in the northern part of the country. Canberra is the capital of the Australian Capital Territory (ACT) and is home to the Australian Bureau of Statistics, the Australian Bureau of Meteorology, and the Australian Institute of Sport. It is also the seat of the Commonwealth Parliament and the Australian Federal Government. Canberra is known for its vibrant culture, natural beauty, and historical significance, making it a popular destination for tourists and locals alike.

Iterative DPO (Exponential Schedule)

The capital city of Australia is Canberra.

Exmaple #7: Countdown Qualitative Samples

Input Prompt

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer.: Using the numbers [60, 17, 14], create an equation that equals 91. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>.Assistant: Let me solve this step by step.

Iterative RLOO (Fixed Schedule)

<think> Let me analyze this step by step:

1. First, let's try to get close to 91 with basic operations: - 60 is already close to 91, we need to add 31 somehow - 17 and 14 could be used to get small numbers that add up to 31
 2. Let's try working with 17 and 14 first: $17 + 14 = 31$ This is exactly what we need! Now we just need to add 60 to get to 91
 3. Let's verify: $17 + 14 = 31$ $31 + 60 = 91$
- This works! We can write it as $(17 + 14) + 60$ </think> <answer> $(17 + 14) + 60$ </answer>

Iterative RLOO (Linear Schedule)

<think> Let me analyze this step by step:

1. First, let's try to get close to 91 with basic operations: - 60 is already close to 91, we need to add 31 somehow - 17 and 14 could be used to get 31
2. Let's try working with 17 and 14 first: - $17 + 14 = 31$ - This is exactly what we need! We can add this to 60
3. Now we need to incorporate 60 somehow: - We can multiply $17 * 14 = 238$ - This is too large, let's backtrack
4. Let's try a different approach: - What if we start with 60? - $60 + 17 = 77$ - $77 + 14 = 91$ - This works! Let's verify: $(60 + 17) + 14 = 91$
5. Let's verify the full sequence: $(60 + 17) + 14 = 77 + 14 = 91$ </think> <answer> $(60 + 17) + 14$ </answer>

References

- Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. 2025. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. <https://doi.org/10.48550/arXiv.2502.02737> arXiv:2502.02737 [cs].
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. <https://doi.org/10.48550/arXiv.2407.21787> arXiv:2407.21787 [cs].
- Yuyan Bu, Liangyu Huo, Yi Jing, and Qing Yang. [n.d.]. Beyond Excess and Deficiency: Adaptive Length Bias Mitigation in Reward Models for RLHF. ([n.d.]).
- Bradley Butcher, Michael O’Keefe, and James Titchener. 2025. Precise Length Control in Large Language Models. *Natural Language Processing Journal* 11 (June 2025), 100143. <https://doi.org/10.1016/j.nlp.2025.100143> arXiv:2412.11937 [cs].
- Weize Chen, Jiarui Yuan, Tailin Jin, Ning Ding, Huimin Chen, Zhiyuan Liu, and Maosong Sun. 2025. The Overthinker’s DIET: Cutting Token Calories with Difficulty-Aware Training. <https://doi.org/10.48550/arXiv.2505.19217> arXiv:2505.19217 [cs].
- Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2024. UltraFeedback: Boosting Language Models with Scaled AI Feedback. <https://doi.org/10.48550/arXiv.2310.01377> arXiv:2310.01377 [cs].

- Mehdi Fatemi, Banafsheh Rafiee, Mingjie Tang, and Kartik Talamadupula. 2025. Concise Reasoning via Reinforcement Learning. <https://doi.org/10.48550/arXiv.2504.05185> arXiv:2504.05185 [cs].
- Kilian Freitag, Kristian Ceder, Rita Laezza, Knut Åkesson, and Morteza Haghir Chehreghani. 2025. Curriculum Reinforcement Learning for Complex Reward Functions. <https://doi.org/10.48550/arXiv.2410.16790> arXiv:2410.16790 [cs].
- Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. 2025. Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs. <https://doi.org/10.48550/arXiv.2503.01307> arXiv:2503.01307 [cs].
- Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D. Goodman. 2024. Stream of Search (SoS): Learning to Search in Language. <https://doi.org/10.48550/arXiv.2404.03683> arXiv:2404.03683 [cs].
- Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2025. Token-Budget-Aware LLM Reasoning. <https://doi.org/10.48550/arXiv.2412.18547> arXiv:2412.18547 [cs].
- Changxin Huang, Guangrun Wang, Zhibo Zhou, Ronghui Zhang, and Liang Lin. 2021. Reward-Adaptive Reinforcement Learning: Dynamic Policy Gradient Optimization for Bipedal Locomotion. <https://doi.org/10.48550/arXiv.2107.01908> arXiv:2107.01908 [cs].
- Zixuan Liu, Xiaolin Sun, and Zizhan Zheng. 2024. Enhancing LLM Safety via Constrained Direct Preference Optimization. <https://doi.org/10.48550/arXiv.2403.02475> arXiv:2403.02475 [cs].
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. <https://doi.org/10.48550/arXiv.2408.03314> arXiv:2408.03314 [cs].
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. <https://doi.org/10.48550/arXiv.2201.11903> arXiv:2201.11903 [cs].
- Heming Xia, Chak Tou Leong, Wenjie Wang, Yongqi Li, and Wenjie Li. 2025. TokenSkip: Controllable Chain-of-Thought Compression in LLMs. <https://doi.org/10.48550/arXiv.2502.12067> arXiv:2502.12067 [cs].
- Tengyu Xu, Eryk Helenowski, Karthik Abinav Sankararaman, Di Jin, Kaiyan Peng, Eric Han, Shaoliang Nie, Chen Zhu, Hejia Zhang, Wenxuan Zhou, Zhouhao Zeng, Yun He, Karishma Mandyam, Arya Talabzadeh, Madian Khabza, Gabriel Cohen, Yuandong Tian, Hao Ma, Sinong Wang, and Han Fang. 2024. The Perfect Blend: Redefining RLHF with Mixture of Judges. <https://doi.org/10.48550/arXiv.2409.20370> arXiv:2409.20370 [cs].
- Heng Yang and Ke Li. 2023. InstOptima: Evolutionary Multi-objective Instruction Optimization via Large Language Model-based Instruction Operators. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Singapore, 13593–13602. <https://doi.org/10.18653/v1/2023.findings-emnlp.907>
- Rui Yang, Xiaoman Pan, Feng Luo, Shuang Qiu, Han Zhong, Dong Yu, and Jianshu Chen. 2024. Rewards-in-Context: Multi-objective Alignment of Foundation Models with Dynamic Preference Adjustment. <https://doi.org/10.48550/arXiv.2402.10207> arXiv:2402.10207 [cs].